JALRAKSHAK AI

Backend + Platform Engineering

Public API, geospatial data platform, real-time events, alerts, routing, incidents, security and deployment

SUBTEAM 2 - BACKEND / PLATFORM (2 MEMBERS)

Member C: API/Data/Geo Lead | Member D: Realtime/Alerts/Routing/Security/DevOps Lead

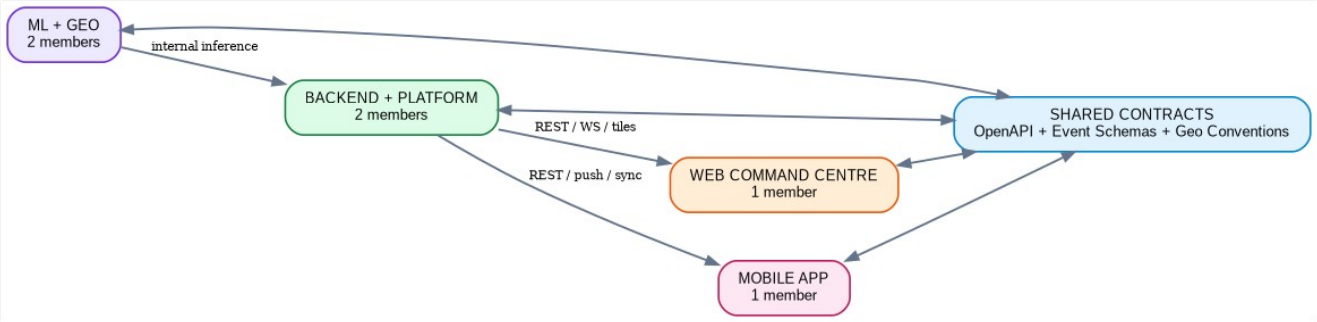
Zero-to-build, full-scope implementation guide | Research verified 06 Sep 2026

Goal: every subteam can build independently against shared contracts, then integrate without rewriting core modules.

Guide Map

Team Split and Independence Contract

Recommended 6-member split: ML/Geospatial 2 | Backend/Platform 2 | Web Command Centre 1 | Mobile App 1



All four subteams share contracts, not implementation details.

The rule that prevents integration chaos

- **Freeze contracts before features.** Create OpenAPI, WebSocket event schemas, GeoJSON/H3 conventions, enums and mock fixtures on day zero.
- **No frontend waits for backend.** Web and mobile develop against generated clients and MSW/mock-server fixtures.
- **No backend waits for ML.** Backend ships an MLStubProvider with deterministic sample nowcast, inundation, risk and report-verification outputs.
- **No ML waits for production data.** ML starts with open/historical/synthetic adapters and exports canonical artifacts; source adapters are replaceable.
- **One public API boundary.** Web and mobile never call ML directly; backend is the public trust/security boundary.
- **Large geospatial data is referenced, not embedded.** Use signed object URLs, COG/Zarr/STAC and vector/raster tiles rather than huge JSON payloads.

Canonical integration conventions

Item	Convention
Time	UTC ISO-8601; source_timestamp + ingested_at + valid_time.
CRS	EPSG:4326 at API boundary; preserve native CRS internally; Web Mercator only for map rendering.
Rainfall	mm/h for rate; mm for accumulation.
Water depth	metres; only emit numeric depth when model is calibrated. Otherwise emit susceptibility/risk class.
Risk levels	LOW, MODERATE, HIGH, SEVERE; confidence is 0.0-1.0.
Spatial key	H3 cell ID + optional ward/admin ID; geometry as GeoJSON/MVT.
Raster	COG for map delivery; Zarr/NetCDF for multidimensional compute; STAC metadata for discovery.
Error envelope	{code, message, trace_id, details}; never expose internal stack traces.
Versioning	/api/v1 public; model_version and data_version on every model result.

Shared Contract Package - Create This First

```
jalrakshak/
  contracts/
    openapi.yaml
    internal-ml-openapi.yaml
    websocket-events.json
    schemas/
      risk-cell.schema.json
      incident.schema.json
```

```
        field-report.schema.json
        alert.schema.json
        model-run.schema.json
      fixtures/
        demo-event/
        risk-cells.json
        incidents.json
        reports.json
        nowcast-manifest.json
      generated/
        typescript-web/
        typescript-mobile/
        python-client/
```

- Backend owns public OpenAPI; ML owns internal inference OpenAPI; changes require one reviewer from the consuming team.
- Generate TypeScript clients from OpenAPI so web/mobile do not hand-write request/response types.
- Store representative GeoJSON, H3, alert, incident and report fixtures in Git so UI work is deterministic.
- Contract tests run in CI before any service can merge to main.

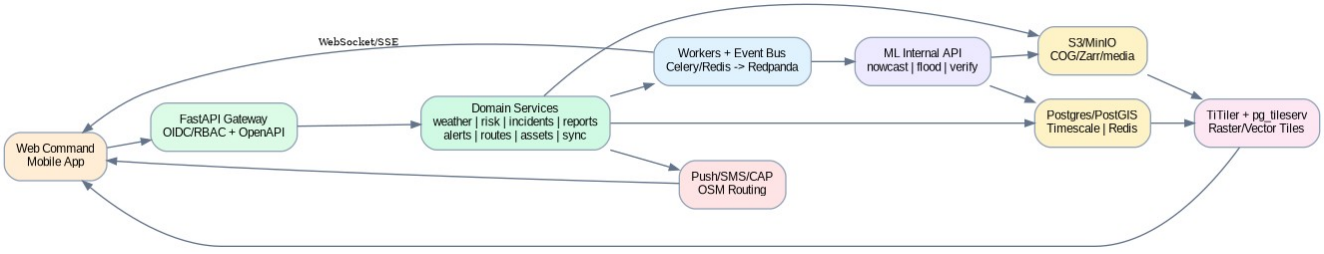
Minimum public endpoint surface

```
GET  /api/v1/weather/current
GET  /api/v1/weather/sources/status
GET  /api/v1/nowcast/manifest
GET  /api/v1/inundation/manifest
GET  /api/v1/risk/cells?bbox=&valid_time=
GET  /api/v1/risk/{h3_cell}/timeline
GET  /api/v1/incidents
POST /api/v1/reports
POST /api/v1/reports/{id}/review
POST /api/v1/routes/lower-risk
POST /api/v1/alerts/draft
POST /api/v1/alerts/{id}/approve
POST /api/v1/alerts/{id}/publish
GET  /api/v1/assets
GET  /api/v1/models/status
POST /api/v1/sync/batch
WS   /api/v1/live
```

Minimum live-event topics

```
risk.cell.updated
incident.created
incident.updated
report.received
report.verified
alert.drafted
alert.published
source.degraded
model.run.completed
field.task.updated
```

Target Backend Architecture



Backend is the public trust boundary and integration hub. ML is internal.

Start as a modular monolith + separate compute/tile services. Do not create 15 microservices before the product works.

Member-level ownership

Member	Primary ownership	Secondary
BE-C	FastAPI, Postgres/PostGIS, Timescale, MinIO, ingestion, tile integration, public OpenAPI	Weather/risk/model APIs, database migrations, contract generation
BE-D	Realtime, incidents, reports, alerts/CAP, routing, watch locations, notifications, responder tasks, audit/security/DevOps	OR-Tools response planning, offline sync, observability, release automation

Environment Bootstrap - From Zero

1. Install Docker, Git and Python 3.12. Use uv/venv for the API package. Use Docker Compose for PostgreSQL/PostGIS, Redis, MinIO, MLflow and optional Redpanda.
2. Start from the shared openapi.yaml and fixtures. The backend must pass contract tests before real ML or real frontend exists.
3. Implement providers behind interfaces: MLProvider, NotificationProvider, RoutingProvider, ObjectStoreProvider. Ship stub providers first.
4. Use database migrations from day one (Alembic). Never manually patch production schema.
5. Run all services locally using one docker compose command and a seeded demo event.

```
backend/
  pyproject.toml
  app/
    main.py
    core/          # config, security, errors, telemetry
    api/v1/        # routers only
    domains/
      weather/
      risk/
      incidents/
      reports/
      alerts/
      routing/
      assets/
      responders/
      replay/
      audit/
      models/
    integrations/
      ml/
      object_store/
      notifications/
      cap/
      tiles/
    db/
    workers/
    realtime/
  migrations/
  tests/
  docker/
```

Recommended Backend Stack

Concern	Primary choice	Reason / alternative
API	FastAPI + Pydantic	OpenAPI-first, async-friendly Python stack; Django/Nest are possible if team prefers.
Relational/geo	PostgreSQL + PostGIS	Single system of record for incidents, assets,

Concern	Primary choice	Reason / alternative
		geometry, H3 and spatial queries.
Time series	TimescaleDB extension or partitioned Postgres	Convenient sensor/risk history; can start with native Postgres partitions.
Cache/queue	Redis + Celery initially	Low operational burden for SIH/full prototype.
Event streaming	Redis Streams initially -> Redpanda when needed	Upgrade path to durable multi-consumer streams; Redpanda is Kafka-client compatible.
Object storage	S3-compatible store / MinIO for local dev	Weather rasters, Zarr/COG/model media do not belong in relational BLOB columns.
Raster tiles	TiTiler	Dynamic tiles from COG/STAC.
Vector tiles	pg_tileserv/PostGIS ST_AsMVT	Efficient H3, roads, incidents and assets.
Observability	OpenTelemetry + Prometheus + Grafana + Sentry	Trace API/worker latency and production failures.

FastAPI async docs: https://fastapi.tiangolo.com/async/ - Use async for I/O-bound routes and clients.
PostGIS manual: https://postgis.net/documentation/manual/ - Spatial queries and raster/vector support.
TiTiler: https://developmentseed.org/titiler/user_guide/dynamic_tiling/ - Dynamic COG tiling.
pg_tileserv: https://access.crunchydata.com/documentation/pg_tileserv/latest/ - PostGIS to Mapbox Vector Tiles.

Core Data Stores and Schema

Table / store	Key fields	Notes
users/profiles	id, role, region_scope, language, notification prefs	Identity claims mirrored from OIDC provider only as needed.
weather_sources	source, status, last_seen, quality, lag_sec	Operational data health.
risk_cells	h3, valid_time, level, probability, confidence, geometry, model_version	Partition/index by time; spatial GIST + H3 indexes.
incidents	id, type, severity, status, geom, started_at, owner, source	Operational lifecycle.
field_reports	id, user/anonymous token, geom, media_uri, depth_category, verification, status	PII minimized; media in object store.
alerts	id, severity, area_geom, valid_from/to, status, content, cap_payload, approver	Draft -> approved -> published state machine.
critical_assets	id, type, name, geom, vulnerability, metadata	Hospitals, schools, shelters, roads, pumps.
responder_tasks	id, incident_id, assigned_to, status, geom, evidence	Offline sync friendly.
audit_log	actor, action, entity, before_hash, after_hash, trace_id, timestamp	Append-only application log.
object store	COG/Zarr/NetCDF/media/replay manifests	Use signed URLs; checksum everything.

Weather/Data Ingestion Platform

- Each source has an adapter interface: list_available(), fetch(), parse_metadata(), normalize(), health_check().
- Ingestion writes raw object first, then normalized artifact. Never overwrite raw source files.
- Store manifest + checksum + provenance in database/STAC. Trigger ML through background job after required inputs for an issue time are ready.
- Graceful degradation: missing radar must not crash the pipeline; emit source.degraded and let ML fusion reduce confidence.
- For demo/historical replay, implement FileSystemDemoAdapter so the entire system works offline with frozen fixtures.

Geospatial Serving Architecture

```
Raster path:
ML output -> Cloud Optimized GeoTIFF in object store -> TiTiler -> XYZ/TileJSON -> Web/Mobile

Vector path:
PostGIS risk/incidents/assets -> pg_tileserv / ST_AsMVT -> MVT -> Web/Mobile

Metadata path:
STAC/manifest -> public API -> client timeline/layer selector
```

- Never send million-cell GeoJSON if MVT or tiles are appropriate.
- Protect private operational layers with signed/tokenized tile requests or a reverse proxy that enforces auth.
- Cache immutable replay tiles aggressively; live risk tiles use short cache TTL and versioned run IDs.

Public API Design

- REST for query/commands, WebSocket/SSE for live deltas, signed URLs for large media/artifacts.
- Use cursor pagination for incidents/reports/audit; bbox/time filters for map data; ETags where useful.
- Every response carries trace_id and server_time. Model-derived responses carry model_version/data_version/valid_time/confidence.
- No frontend-specific database shapes. Use stable domain DTOs.

```
# standard error
{
  "code": "SOURCE_DEGRADED",
  "message": "Radar source is delayed; forecast confidence reduced.",
  "trace_id": "...",
  "details": {"source":"imd_radar","lag_seconds":840}
}
```

ML Orchestration

- Backend calls the internal ML client; it never imports torch/model code.
- Long-running inference/training is asynchronous. API creates a run record -> worker invokes ML -> artifacts are stored -> risk materialized -> live event emitted.
- Use idempotency key per issue_time/model/data manifest so duplicate jobs do not create conflicting outputs.
- Fallback provider returns deterministic fixtures when ML is unavailable, allowing frontend/mobile demos to continue.

Async Jobs and Event Bus

- Celery handles ingestion, media verification orchestration, alert fan-out and replay preprocessing in the initial architecture.
- Redis is acceptable for prototype task queues; RabbitMQ is stronger for durable production task delivery. Keep broker configurable.
- For multi-consumer event history, move live domain events to Redis Streams or Redpanda. Redis docs explicitly distinguish durable Streams from fire-and-forget Pub/Sub.
- Use outbox pattern for critical events: write DB transaction + outbox row, then publish. This avoids lost alert/incident events.

Celery: https://docs.celeryq.dev/en/stable/getting-started/first-steps-with-celery.html - Task queue and broker concepts.

Redis Streams: https://redis.io/docs/latest/develop/data-types/streams/ - Durable ordered event stream.

Redpanda: https://docs.redpanda.com/streaming/current/develop/kafka-clients/ - Kafka-client compatible scale option.

Authentication and Region-Scoped RBAC

Role	Capabilities
Citizen	Read public risk/alerts, manage watch locations, submit reports.
FieldResponder	Citizen + assigned tasks, road/drain status, verified field evidence.
Analyst	Operational layers, incidents, reports, model/data health, draft alerts.
AlertApprover	Analyst + approve/publish alerts inside authorized region.

Role	Capabilities
MunicipalOfficer	Response planning/assets/task allocation for region.
Admin	Users, region scopes, source configuration, integrations.
MLAdmin	Model status/deployment metadata; cannot publish alerts unless separately authorized.

- Use OIDC/OAuth2. MVP can use Supabase Auth; enterprise/on-prem can use Keycloak or organizational identity provider.
- Enforce permissions server-side and at row/region scope. UI hiding is not authorization.
- All privileged mutations require audit records.

Weather, Risk and Model Operations APIs

- Weather service exposes current source status, latest observation/forecast manifests and freshness.
- Risk service returns H3/ward rankings, timeline, critical-exposure summary and layer manifests.
- Models service proxies ML registry/run health; never exposes raw credentials or internal storage paths.
- Data-health endpoint aggregates lag, missing percentage, quality and degraded fallback state.

Incident Management

```
DETECTED -> OPEN -> ACKNOWLEDGED -> MITIGATING -> RESOLVED -> CLOSED
                \-> DISMISSED (with reason + audit)
```

- Create incidents from severe risk, verified reports or manual analyst action; source must be explicit.
- Timeline is append-only domain events: risk changed, report verified, responder assigned, road closed, alert published.
- Link multiple reports and assets to one incident rather than creating duplicates for every signal.

Field Report Workflow

1. Client requests signed upload URL and creates draft report metadata.
2. Client uploads media directly to object storage; backend finalizes report with checksum.
3. Worker invokes ML verification; backend applies policy thresholds and human-review rules.
4. Verified/likely report appears on operational map and may influence risk according to policy; rejected media never silently deletes the audit record.
5. Responder/analyst can override with reason; override is audited.

Alert and CAP Workflow

JalRakshak should generate CAP-compatible structured alerts for authorized dissemination. It should not claim to replace NDMA SACHET.

```
AI/Rules recommend -> Analyst drafts -> Approver reviews -> Publish -> Fan-out -> Delivery receipts
```

Alert field	Backend rule
severity	Derived recommendation, editable only by authorized role.
area	Polygon/H3/admin region; validate it is within approver scope.
validity	effective + expires; reject inverted/expired windows.
impact/action	Template-driven; multilingual variants versioned.
confidence	Expose source/model confidence separately from official severity.
CAP	Serialize to CAP 1.2-compatible payload where integration is enabled.

NDMA SACHET: https://sachet.ndma.gov.in/ - CAP-based geo-targeted, multilingual alert ecosystem.

OASIS CAP 1.2: https://docs.oasis-open.org/emergency/cap/v1.2/CAP-v1.2-os.html - Common Alerting Protocol standard.

Lower-Risk Routing Service

- Build on OpenStreetMap graph using OSMnx/NetworkX for prototype or OSRM/Valhalla/GraphHopper for routing engine scale.
- Edge cost = travel cost + flood-risk penalty + depth penalty + closure penalty + uncertainty penalty.
- Hard-closed roads get infinite/unusable cost; severe uncertain segments are penalized, not advertised as guaranteed safe.
- Return route geometry plus reasons for avoided segments and data timestamp.

Watch Locations and Notifications

- Store Home/Family/College/Office labels as user-owned points; mobile can keep private display labels local if preferred.
- Subscribe watch locations to H3/admin zones. When risk crosses threshold or official alert area intersects, enqueue push notification.
- Support quiet-hour policy only for non-emergency informational messages; severe safety alerts follow policy/authority requirements.
- Push provider interface enables Expo Push initially and FCM/APNs direct later. SMS/WhatsApp must be approved integrations, not hard-coded demo claims.

Field Responder Tasks

- Tasks: verify location, measure depth, mark road blocked, drainage obstruction, rescue/support request, reopen road, close task.
- Use optimistic locking/version numbers because mobile can sync offline updates later.
- Keep task evidence/media separate from citizen public reports; responder identity is protected by role authorization.

Response Resource Optimization

- Use OR-Tools as a recommendation engine for pumps/teams/shelters under capacity, travel-time and priority constraints.
- Inputs are explicit and editable; output is a suggested assignment plan with objective value and constraint violations.
- Never auto-dispatch. Human approval creates responder tasks and audit records.

Historical Replay Service

- A replay is an immutable manifest of timestamped weather, risk, incident, report and alert snapshots.
- Serve a virtual clock so the web can scrub through an event without mixing live data.
- Precompute/copy immutable tile manifests for deterministic judge demos.
- Expose predicted-vs-observed metrics from ML reports at replay end.

Offline Sync API

```
POST /api/v1/sync/batch
{
  "device_id":"...",
  "cursor":"...",
  "mutations":[
    {"client_id":"uuid","type":"REPORT_CREATE","created_at":"...","payload":{...}},
    {"client_id":"uuid","type":"TASK_UPDATE","base_version":7,"payload":{...}}
  ]
}
```

- Server de-duplicates by client_id and returns per-mutation status.
- Conflicts are explicit: accepted, duplicate, rejected, needs_merge. Never silently overwrite responder updates.
- Client receives new cursor + server deltas for cached alerts/tasks/watch locations.

Realtime API

- WebSocket for command centre live state; mobile can use WebSocket while foregrounded and push/background sync otherwise.
- Events are small deltas containing entity IDs/version/run IDs. Clients refetch full detail through REST if needed.
- Heartbeat, reconnect with backoff, last_event_id replay and authorization are mandatory.

Audit, Security and Privacy

- TLS everywhere; secure headers; request size/rate limits; signed upload URLs; MIME validation; malware/content scanning hook; no public bucket.
- PII minimization: citizen map should not expose reporter identity or exact private profile information.
- Secure secrets with environment/secret manager; never commit keys.
- Append-only audit for alert publication, model version activation, incident override, responder status, report review and admin changes.
- Every model result stores model_version/data_version; every alert stores which risk snapshot supported it.

Observability

Signal	Examples
Metrics	API p95 latency, source lag, queue depth, inference duration, tile latency, push delivery, sync conflicts.
Traces	request -> DB -> worker -> ML -> object store -> event publish.
Logs	structured JSON with trace_id, actor_id/role (non-sensitive), entity ID, status.
Alerts	radar stale, ingestion failed, worker backlog, ML unavailable, DB saturation, push failure spike.

OpenTelemetry Python: https://opentelemetry.io/docs/languages/python/instrumentation/ - Distributed traces/metrics/log instrumentation.

Prometheus + Grafana: https://prometheus.io/docs/visualization/grafana/ - Operational monitoring dashboards.

Local and Production-like Deployment

```
docker compose services (development)
  api
  worker
  scheduler
  postgres-postgis
  redis
  minio
  titiler
  pg_tileserv
  ml-stub or ml-service
  prometheus
  grafana
  optional: redpanda + console
```

- Use Kubernetes only after Compose is stable; SIH does not gain points from unused Kubernetes YAML.
- Production-like reverse proxy terminates TLS and routes API/tiles/WebSocket.
- CI: lint -> unit -> contract -> DB migration test -> integration -> Docker image -> vulnerability scan -> deploy preview.

Testing Strategy

Test	Must prove
Unit	domain state machines, risk access rules, route penalty, CAP serializer, sync conflict logic.
Contract	OpenAPI requests/responses match generated clients and ML internal schemas.
Integration	PostGIS spatial queries, MinIO signed uploads, Celery worker, tile servers, ML stub/real service.
Security	RBAC/region boundaries, object access, rate limit, malformed media, privilege escalation attempts.

Test	Must prove
Replay	same frozen manifest produces deterministic sequence of public events.
Load	risk-cell/tile endpoints and WebSocket fan-out remain responsive under demo/target concurrency.

Independent Development Plan

Sprint	BE-C	BE-D	Integration artifact
0	OpenAPI + DB bootstrap + ML stub	Auth/RBAC + event schemas + notification stub	Seeded compose + fixtures
1	Weather/risk storage + tiles	Incidents/reports + live events	Web/mobile mock parity
2	Ingestion + model run orchestration	Alerts/CAP + watch notifications	End-to-end fake event
3	Real ML client + geospatial hardening	Routing + responder tasks + offline sync	Mobile/Web integration
4	Historical replay + model/data health	OR-Tools + audit + security	Full product demo flow
5	Performance/cache/indexing	Observability/release hardening	Production-like compose/deploy

Complete Definition of Done - Backend/Platform

- [ ] OpenAPI/contract generation
- [ ] Postgres/PostGIS migrations
- [ ] time-series risk/weather history
- [ ] S3/MinIO object layer
- [ ] weather/source ingestion adapters
- [ ] STAC/manifest metadata
- [ ] TiTiler raster tiles
- [ ] pg_tileserv/vector tiles
- [ ] ML stub + real internal client
- [ ] async workers
- [ ] event bus/live feed
- [ ] OIDC auth + region RBAC
- [ ] weather/risk/model APIs
- [ ] incident lifecycle/timeline
- [ ] citizen field-report upload + verification orchestration
- [ ] human review/override
- [ ] alert draft/approve/publish state machine
- [ ] CAP 1.2 serialization
- [ ] push notification fan-out
- [ ] watch locations
- [ ] lower-risk routing
- [ ] responder tasks/evidence
- [ ] OR-Tools response recommendation
- [ ] historical replay clock/manifests
- [ ] offline sync/conflict handling
- [ ] audit log
- [ ] security/rate limits/signed URLs
- [ ] OpenTelemetry/Prometheus/Grafana/Sentry hooks
- [ ] Docker Compose full stack
- [ ] CI tests + seed/demo scripts

Research and Implementation Resources

FastAPI: https://fastapi.tiangolo.com/

PostGIS: https://postgis.net/documentation/manual/

Celery: https://docs.celeryq.dev/en/stable/
Redis Streams: https://redis.io/docs/latest/develop/data-types/streams/
Redpanda: https://docs.redpanda.com/streaming/current/
TiTiler: https://developmentseed.org/titiler/
pg_tileserv: https://access.crunchydata.com/documentation/pg_tileserv/latest/
NDMA SACHET: https://sachet.ndma.gov.in/
OASIS CAP 1.2: https://docs.oasis-open.org/emergency/cap/v1.2/CAP-v1.2-os.html
OpenTelemetry: https://opentelemetry.io/docs/languages/python/